

Mono-fix or Poly-fix? On the Prevalence and Localizability of Multi-Language Bug Fixes

Eashaan Rao and Raghavendra | FSE 2027, with ICPC 2027 and IST as fallbacks | Rebuilt after SANER 2026 Registered Report rejection

Core Argument. A bug fix frequently touches more than source code in the project's main language. It touches configuration, test data, schemas and build files. This paper reports how often that happens and what state-of-the-art localization models do when it does. The claim the paper is written to support is stronger than a performance gap: a class of fix exists that some models cannot process at all, because the files are not parseable by the architecture, while classical information retrieval handles them without difficulty. If that holds, the field's measured progress over classical baselines may hold only on the subset of fixes its models are able to ingest.

The Six Questions

1. What is the problem?

Bug localization is evaluated on the assumption that a fix is contained within source files of a project's main language. Real fixes are often not. Correcting a defect may require changing the source, updating a test, and revising a schema or a configuration file that encodes the expected behaviour.

Whether localization techniques handle such fixes is unknown, because the question has not been asked. Existing characterisations of bug fixes measure how many files a fix touches and how far apart those files sit in the directory hierarchy. None measures what kind of file is touched.

Two things could go wrong when a fix crosses file types, and they are different in kind. A model may be unable to process the file at all, since techniques that parse source into abstract syntax trees have no parser for a configuration or data file. Or a model may process the file and fail to rank it, since the vocabulary and structure of such files differ from what the model was trained on. The first is an architectural limit and the second is a semantic one, and no existing evaluation separates them.

2. Why is it important?

If the architectural limit is real, it is more consequential than a performance gap. A model that ranks a file poorly can be improved. A model that cannot read the file has no path to the answer, and no amount of training data changes that.

The consequence for the field is specific. Classical information retrieval treats every file as a bag of tokens and can therefore process any file type. The techniques that cannot are precisely the ones that parse structure, which is what distinguishes them from classical baselines in the first place. If those techniques fail on fixes that cross file types, then reported improvements over classical retrieval may hold only on the portion of the task the newer techniques are able to ingest.

That is a question about what a decade of reported progress means, and it is answerable with a measurement rather than an argument.

3. What are the existing solutions, and why are they not enough?

Characterisation of bug fixes. Recent work classifies multi-hunk patches by spatial proximity, distinguishing hunks within one method, within one file across methods, and across files within a package, and reports that the median number of files modified per bug does not exceed two. At ICPC 2026, work on agent-based localization reports performance broken down by the number of ground-truth documents, showing a decline beyond four. Both measure quantity and distance. Neither measures file type.

Cross-language bug localization. BLAZE performs localization across projects written in different languages, supported by BeetleBox, a dataset spanning five languages. The unit remains a single-language fix within a project of that language. xLoc detects and localizes bugs arising from interactions between languages, which is a different phenomenon: a bug caused by a language boundary rather than a fix that crosses one.

Dataset quality in bug localization. An established line examines whether ground truth is correct, covering misclassified reports, already-localized reports and incorrect ground truth, with one study reporting that average precision changed for approximately 90 percent of reports linked to incorrect ground truth. All of this concerns files wrongly included. This paper concerns files that fixes contain and evaluations may not reach, which is the opposite direction and is not covered.

Polyglot software engineering. Studies establish that most projects use several languages and that cross-language links are a source of defects. Recent testing work shows regression test selection tools can be unsafe on polyglot projects because inter-language dependencies are missed. That precedent is close in spirit and concerns a different task.

The common weakness: the localization literature has characterised fixes by size and dispersion but not by composition, and the polyglot literature has examined language interaction but not localization. The strengths we use: BeetleBox provides a multi-language dataset with fix ground truth, and the multi-hunk characterisation work provides a methodological template, since classifying by file type is the typological counterpart to their spatial classification.

4. What is the proposed idea?

An exploratory study in four steps, each answerable independently, structured so that a null result at any step is reportable rather than fatal.

The unit is the fix, not the project. A bug is Mono-fix if every file in its fix belongs to one language, and Poly-fix otherwise. The classification is made from the fix commit and does not depend on the project's language composition. This is a deliberate change from the earlier design, where projects were classified using a byte-count threshold, which permitted a project labelled monolingual to contain fixes that were not.

Step one, prevalence. Across the dataset, what proportion of fixes are Poly-fix, and what file types do the additional files belong to. This is descriptive, it requires no model, and it determines whether the remaining steps are worth running.

Step two, architectural feasibility. For each model, can it process a Poly-fix bug at all. Techniques that parse source into structured representations are expected to fail on files with no available parser. The outcome is binary per model per file type and is reported as such rather than as a score.

Step three, localization performance. For models that can process the files, how does performance differ between Mono-fix and Poly-fix bugs, controlling for project by comparing within the same project.

Step four, the classical baseline. Classical information retrieval processes every file type by construction. Where the parsing models fail or degrade, does classical retrieval hold. This is what turns a performance observation into a claim about the field's measured progress.

What is different. Fixes are classified by composition rather than by size or dispersion. Architectural capability is separated from semantic performance, so a model that cannot read a file is not reported as a model that ranked it poorly. And the comparison against classical retrieval is framed as a test of whether reported advances hold across the whole task or only across the part that parses.

5. Why should we believe it?

The study is exploratory and is stated as such. Research questions are descriptive, hypotheses where stated are predictions rather than confirmatory tests, and no claim is made about statistical significance except where a test is actually performed.

Classification is mechanical, decided from file extensions in the fix commit, so it is reproducible without annotator judgment for the common cases. Boundary cases must be enumerated in advance: generated files, vendored dependencies, lock files, documentation, and test fixtures. Each is included or excluded by a stated rule, and the proportion affected by each rule is reported so a reader can see how much the definition carries.

Assumptions to state. That file extension determines language, which holds for the file types in scope and should be stated rather than assumed. That the fix commit is the fix, which inherits the tangling problem the dataset literature has documented, where commits combine bug fixing with refactoring. And that architectural failure observed on one implementation reflects the technique rather than the implementation, which requires reporting the version used and whether failure is inherent to the approach or specific to the available code.

The principal threat is data leakage and it must be addressed, not noted. BeetleBox was constructed alongside BLAZE, which is a candidate model. Evaluating BLAZE on BeetleBox confers an advantage no other model has. Either BLAZE is excluded, or its results are reported separately and never aggregated with the others, and the choice is stated in the method.

A second threat is that the retrieval stage bounds what any reranker can achieve, so a reranker cannot be credited for failing to rank a file the retriever never returned. Retrieval recall must be reported as a property of the setup so that reranker results are interpretable, and it should not be presented as a research question.

A third is scope. Two languages, one dataset. The result describes what it describes and generalisation should be claimed narrowly.

6. Where are the artifacts?

The Mono-fix and Poly-fix classification for every bug in the dataset, with the rule applied to each boundary case. The scripts that produce it from fix commits. The per-model feasibility results, recording which models could process which file types. The performance results by class. And the analysis scripts.

The labelled classification is the reusable asset. No dataset records which bugs have fixes crossing file types, and releasing it would let any future technique report class-separated results without repeating the classification.

What This Stands On

Field	Detail
Base paper	Chakraborty, P., Alfadel, M., and Nagappan, M. BLAZE: Cross-language and cross-project bug localization via dynamic chunking and hard example learning. IEEE Transactions on Software Engineering, 2025. Relationship: extension, and also the source of the dataset. They establish cross-language localization, meaning a technique that works across projects written in different languages, and release BeetleBox to support it. We extend the question from projects that differ in language to fixes that span languages, which their design does not address and their dataset was not built to expose. The dependence on their dataset is also the study's principal validity threat and is handled explicitly.
Theoretical basis	Partial, and the brief states this rather than manufacturing one. The operative distinction is between architectural capability and semantic performance: whether a technique can represent an input at all is prior to how well it ranks it, and evaluations that report only ranking metrics cannot distinguish the two. This is a familiar distinction in program analysis, where parseability bounds what any analysis can consider, and it is what the study operationalises. It is a real basis and a modest one. The alternative, framing this through information retrieval's assumption of a homogeneous document collection, was considered and is weaker, since it justifies the unit of analysis rather than the finding.
Empirical standards	Repository Mining (primary, for the classification and prevalence), Benchmarking, since results are reported on an existing dataset, Data and artifact standard, for the released classification, Sampling supplement, for project and bug selection

Mapping to the SIGSOFT Empirical Standards

Standard	Why it applies	Where the work currently falls short
Repository Mining	Primary. The classification and prevalence results are derived from fix commits across a repository corpus.	Requires the extraction procedure, the classification rules and their boundary cases to be documented sufficiently for reproduction. The earlier submission was criticised for exactly this: terms such as valid and extension were used without definition. Every rule must be stated before it is applied.
Benchmarking	All model results are reported on an existing dataset.	Requires the dataset's construction and limitations to be described, and requires that results not rest on a benchmark advantaging one of the evaluated systems. BeetleBox was built alongside BLAZE, which is why BLAZE is either excluded or reported separately.
Sampling supplement	Applies to which projects and bugs are analysed, and to the exclusion of languages present in the dataset.	Not yet stated. The earlier submission excluded C++ without explanation and reviewers asked whether this was a technical constraint or a design choice. Both the exclusion and its rationale must be recorded.
Data and artifact standard	The classification and scripts are released.	Requires documentation sufficient for independent reuse, including the boundary-case rules, since the classification is only reusable if others can see what was counted.

Not applicable, despite appearances. Experiments with Human Participants does not apply, since no participants are involved and no annotation study is conducted. Registered Reports does not apply to this submission, and the earlier registered report is superseded rather than revised. Stating this matters because a reader aware of the SANER submission may expect a confirmatory design, and this study is exploratory by intention.

How It Will Be Evaluated

Metric	Measures	Why this construct	Source	What counts as good
Poly-fix prevalence	Proportion of bugs whose fix touches files of more than one language, and the distribution of additional file types	The first result and the gate on everything else. If Poly-fix bugs are rare, the study's premise does not hold and the paper should say so	New, defined in this work, computed from fix commits	Descriptive. A low prevalence is a publishable negative result about the phenomenon and must be reported rather than reframed. A substantial prevalence motivates the remaining steps
Architectural feasibility per model per file type	Whether a model can process a file of a given type at all, recorded as success or failure with the failure mode	The paper's load-bearing result. A model that cannot ingest an input is categorically different from one that ranks it poorly, and no existing evaluation separates them	New, defined in this work	Binary and reported as such. Failures must be attributed to the technique or to the implementation, since a missing parser in available code is not the same as a technique that cannot represent the input
Mono-fix against Poly-fix performance, within project	Standard localization metrics computed separately by class, compared within the same project	Controls for project-specific factors so that the comparison isolates fix composition rather than project characteristics	Standard localization metrics, applied to a new partition	Descriptive, with effect size reported. No difference is a publishable result and would mean the phenomenon exists without consequence
Classical retrieval against parsing models on Poly-fix	Performance of a classical information retrieval baseline compared with models that parse structure, on Poly-fix bugs specifically	This is what converts a performance observation into a claim about the field. Classical retrieval processes any file type by construction, so it is the condition under which the parsing models' advantage can be tested	Standard baselines, applied to the new partition	Any inversion, where classical retrieval matches or exceeds the parsing models on Poly-fix while trailing on Mono-fix, is the paper's strongest possible finding. No inversion means the paper is a performance study and the venue should change
Boundary case sensitivity	How the prevalence figure changes under alternative treatments of generated files, vendored dependencies, lock files and documentation	The headline number depends on definitional choices, and a reader must be able to see how much	New, defined in this work	Reported as a range rather than a single figure where the choice materially changes the result

Metric	Measures	Why this construct	Source	What counts as good
Retrieval recall	Proportion of ground-truth files reachable by the retrieval stage, by class	Bounds what any reranker can achieve. Reported as a property of the setup rather than as a research question, since the earlier submission was criticised for framing it as one	Standard retrieval practice	Reported per class. If recall differs between Mono-fix and Poly-fix, that difference is itself informative and must not be conflated with reranker performance

Extension Delta

Relationship to the SANER 2026 registered report submission, which was rejected with two reject and one borderline. The idea was not rejected: two reviewers described the topic as interesting and relevant, and one accepted the premise entirely. The objections were to the protocol. Four changes follow directly from them. The unit moves from project to fix, which removes the byte-threshold objection, where a project classified as monolingual was shown to contain Poly-fix bugs. The retrieval ceiling question is removed as a research question and becomes a reported property of the setup, since two reviewers found it incoherent given that ground truth files were force-included. The study is declared exploratory rather than confirmatory, which resolves the objection that a controlled experiment was the wrong design for descriptive questions. And the data leakage between BeetleBox and BLAZE, raised by one reviewer and unaddressed in the submission, is handled in the method rather than in threats.

A fifth change is not a design matter but must be stated. The submission was criticised repeatedly for terms used without definition and for claims stronger than the evidence, including a first comprehensive investigation and a promise to reveal key limitations before any result existed. The register of the paper must change: descriptive verbs, defined terms before use, and no adjective attached to a number.

State of the Art and References

Checked 6 September 2026.

What is occupied. Multi-hunk and multi-file fix characterisation, by spatial proximity, with the finding that the median number of files per fix does not exceed two. Performance by ground-truth document count, reported at ICPC 2026, showing decline beyond four documents. Cross-language localization, by BLAZE with BeetleBox. Bugs arising from language interaction, by xLoc. And dataset quality critique, an established line covering misclassified reports, already-localized reports and incorrect ground truth, with one result finding that incorrect ground truth does not significantly affect localization results.

What is open. Classification of fixes by composition rather than by size or dispersion. The separation of architectural capability from semantic performance. And whether classical retrieval holds where parsing models cannot reach, which is the paper's strongest available claim.

What this means for framing. The dataset quality line is close enough that a reviewer will raise it, and the distinction must be made early: that work concerns files wrongly present in ground truth, this concerns what fixes contain. The ICPC 2026 result is useful rather than threatening, since it establishes at the target venue that the field measures how many files and not what kind.

What remains for the students. The exhaustive pass and the literature sheet. Particular attention to any 2026 work classifying fixes by file type, which is the direct collision risk, and to the polyglot testing literature, since regression test selection has already established that monolingual tools can be unsafe on polyglot projects and that precedent is the closest argument by analogy available.

Technical Tasks

Task	What to produce	Done when	Cost
Extract fix commits for every bug in the dataset and record the complete set of files changed, before any filtering.	A table of bug identifier against changed files with their extensions.	The file count per bug is recoverable and reconciles with the dataset's own ground truth where the two overlap. Any bug whose commit cannot be recovered is recorded rather than dropped silently.	Low
Write the classification rules, including every boundary case, before classifying anything. Generated files, vendored dependencies, lock files, documentation, test fixtures and build configuration each get an explicit rule.	The rule set, with a stated decision and a rationale for each category.	Two people independently apply the rules to a sample of fifty bugs and reach the same classification. Disagreements are resolved by amending the rule, not by adjudicating the case.	Low
Classify every bug as Mono-fix or Poly-fix and compute prevalence, with the distribution of additional file types.	The prevalence figure, the file type distribution, and the labelled classification for the whole dataset.	The number exists. This is the gate: bring it to Ch before proceeding, since a low prevalence changes the paper.	Low
Compute the sensitivity of prevalence to each boundary rule.	Prevalence under alternative treatments of each contested category.	The range is known and it is clear which rules materially affect the headline.	Low
Select the models. One classical information retrieval baseline, one language-agnostic neural model, one model that parses source into a structured representation. Record versions and sources.	The three models running on a small sample, with versions pinned.	Each runs end to end on a Mono-fix bug and produces a ranking. Any model that cannot be made to run is documented with the reason rather than substituted quietly.	Medium
Test architectural feasibility. For each model, attempt to process files of each type present in Poly-fix bugs, and record success, failure, and the failure mode.	A matrix of model against file type with outcome and failure mode.	Every cell is filled. Failures are attributed either to the technique or to the available implementation, and the distinction is stated.	Medium
Build the retrieval stage and report recall by class, as a property of the setup.	Retrieval recall for Mono-fix and Poly-fix separately.	Recall is known for both classes before any reranking result is computed, so reranker results can be interpreted against it.	Medium
Run the within-project comparison. For each project containing both classes, compute localization metrics separately by class.	Per-project, per-class results for each model that can run.	Every project reported contains both classes, so the comparison is within-project rather than across projects with different compositions.	Medium
Handle the BLAZE and BeetleBox relationship as decided. Either exclude BLAZE or report it separately and never aggregate it with the others.	The decision recorded in the method, and results presented accordingly.	No aggregate figure includes BLAZE if BLAZE is retained.	Low

Task	What to produce	Done when	Cost
Compare classical retrieval against the parsing models on Poly-fix bugs specifically.	The comparison, per class, with effect sizes.	The comparison is computable and the direction is clear. If no inversion appears, report it and tell Ch, since the venue decision depends on it.	Low

Research Tasks

1. Read the three SANER reviews closely before anything is written, and produce a table of every objection against how the redesign addresses it. Objections that are not addressed should be listed as such rather than omitted.
2. Settle the premise question in one paragraph. Establish, from the localization literature and from existing benchmarks' own ground truth, whether non-source files appear as localization targets. One reviewer said he would not expect a tool to identify test files, and that objection must be answered rather than left. If the answer is that they do not, the framing narrows to what fixes contain rather than what tools should find.
3. Write the classification rules with Ch before applying them, since the rules carry the headline number and every reviewer objection about definitions traces to rules being implicit.
4. Position against the multi-hunk characterisation work explicitly. They classify by spatial dispersion, this classifies by composition, and the parallel should be stated rather than left for a reviewer to notice.
5. Position against BLAZE and xLoc precisely. BLAZE handles projects in different languages, xLoc handles bugs arising from language interaction, and neither addresses a fix that spans languages. This distinction is the paper's novelty claim and it must be exact.
6. Position against the dataset quality literature, which concerns files wrongly included in ground truth. This work concerns composition, which is a different direction. One study in that line reports that incorrect ground truth does not significantly affect results, and a reviewer will raise it, so it should be addressed before they do.
7. Decide and document the language scope, including any exclusion, with the reason stated as a technical constraint or a design choice.
8. Fix the register before drafting. Descriptive verbs, no first comprehensive, no claims about what results will reveal, terms defined before use, and no adjective attached to a number. The earlier submission failed on this across all three reviews and new results will not repair it.
9. Run the exhaustive literature pass and produce the sheet with the strength-we-use and weakness-we-address columns.
10. Draft threats to validity early, covering data leakage, commit tangling, extension-based language attribution, implementation against technique in feasibility failures, and scope.

Ordering and Dependencies

The classification rules gate everything, because prevalence, the class partition and every downstream comparison depend on them. They must be written and agreed before any bug is classified.

Prevalence gates the study. If Poly-fix bugs are rare the paper changes shape, and this is known after three cheap tasks rather than after the model runs.

Architectural feasibility and the classical baseline comparison together gate the venue. If parsing models fail where classical retrieval holds, the argument is strong enough for FSE. If they merely degrade, the paper is a performance study and belongs at ICPC or IST. Both are known before the full runs are complete, so the venue decision should follow them rather than precede them.

Retrieval recall must be computed before reranker results are interpreted, not after, since a reranker cannot be credited or blamed for a file the retriever never returned.

The premise question should be settled before the introduction is drafted, since it determines whether the paper claims tools ought to find these files or only reports what happens when they meet them.

Decisions for Ch

1. Venue, once feasibility and the baseline comparison are known. FSE 2027 closes 2 October, ICPC 2027 requires a mandatory abstract on 29 October with the paper on 5 November, and IST has no deadline. The ladder is FSE, then ICPC, then IST, but the FSE attempt is only worth making if the architectural finding holds.
2. Whether BLAZE is excluded or reported separately. Excluding it loses a strong model; retaining it requires that no aggregate figure includes it.
3. Who writes. The SANER reviews were about writing rather than about work, and the same document with new numbers would draw similar comments. If Eashaan writes, the register rules in the research tasks are the priority. If the framing sections move elsewhere, that should be deliberate.
4. Whether the premise is defended or the framing narrowed, once the premise question is settled.
5. Whether Objective 1 going to JSS affects this paper's journal fallback, since two submissions to the same journal from one thesis is a pattern reviewers notice.

References

1. Chakraborty, P., Alfadel, M., and Nagappan, M. BLAZE: Cross-language and cross-project bug localization via dynamic chunking and hard example learning. *IEEE Transactions on Software Engineering*, 2025.
2. Learning to detect and localize multilingual bugs (xLoc). *Proceedings of the ACM on Software Engineering*, 2024.
3. Characterizing multi-hunk patches: Divergence, proximity, and LLM repair challenges. *arXiv:2506.04418*.
4. Improved bug localization with AI agents leveraging hypothesis and dynamic cognition. *ICPC 2026*.
5. Kochhar, P. S., Tian, Y., and Lo, D. Potential biases in bug localization: Do they matter? *ASE 2014*, 803 to 814.
6. Are datasets for information retrieval-based bug localization techniques trustworthy? *Empirical Software Engineering*, 2021.
7. Niu, F., Li, C., Liu, K., Xia, X., and Lo, D. When deep learning meets information retrieval-based bug localization: A survey. *ACM Computing Surveys*, 57(11), 2025.
8. Tomassetti, F., and Torchiano, M. An empirical assessment of polyglot-ism in GitHub. *EASE 2014*.
9. Ralph, P., Hoda, R., and Treude, C. *ACM SIGSOFT Empirical Standards*. ACM, 2020. <https://www2.sigsoft.org/EmpiricalStandards/>

AI Disclosure

The original ideas, analysis, and research synthesis in this report are the intellectual contribution of the authors. AI-assisted tools were used to enhance the clarity and presentation of the writing.